

PROJECT REPORT

Automata Visualizer

An Interactive DFA/NFA Simulation & Visualization Tool

1. Project Members

Sr.	Name	Roll No.
1	Christina Esther Nadar	23108A0009
2	Nandini Yedelli	23108A0038
3	Samiksha Chavan	23108A0057

2. Project Title

Automata Visualizer — An Interactive Web-Based Tool for DFA/NFA Simulation, NFA to DFA Conversion, and DFA Minimization

3. Problem Statement

Theory of Computation is a foundational subject in Computer Engineering, yet students frequently struggle to grasp abstract concepts such as finite automata, state transitions, NFA-to-DFA conversion via subset construction, and DFA minimization using partitioning. Traditional text-book representations fail to make these dynamic, step-by-step processes visually intuitive.

There is a clear need for an interactive, browser-based tool that allows students and educators to:

- Define and visualize Deterministic Finite Automata (DFA) and Non-Deterministic Finite Automata (NFA) using the formal 5-tuple representation $(Q, \Sigma, \delta, q_0, F)$.
- Animate state transitions as an input string is processed character by character.
- Automatically convert an NFA to an equivalent DFA through subset construction.
- Minimize a given DFA using the table-filling (partitioning) method.
- Immediately observe the acceptance or rejection result for any tested string.

Without such a tool, students must manually trace transitions on paper, a tedious and error-prone process that discourages deep understanding.

4. Proposed Methodology

The project follows an incremental development methodology divided into four functional modules:

4.1 State Machine Input (5-Tuple Representation)

Users define the automaton using the formal 5-tuple $(Q, \Sigma, \delta, q_0, F)$:

- Q — finite set of states (entered as state names, e.g., q_0, q_1, q_2)
- Σ — input alphabet (e.g., $\{0, 1\}$ or $\{a, b\}$)
- δ — transition function (defined via a transition table in the UI)
- q_0 — start state (designated with an arrow)
- F — set of accepting/final states (double-circled in the diagram)

The canvas engine renders each state as a circle, transitions as directed arcs with labels, self-loops where required, and distinguishes start and accept states visually.

4.2 NFA to DFA Conversion (Subset Construction)

For an NFA input, the tool applies the subset construction algorithm:

- Computes the ϵ -closure of each reachable state set.
- Iteratively builds new DFA states as subsets of NFA states.
- Animates each new subset state being discovered and added to the transition table.
- Highlights the final DFA states corresponding to subsets containing NFA accept states.

4.3 DFA Minimization (Partitioning Method)

Given a DFA, the minimization module:

- Initialises two partitions: accepting states and non-accepting states.
- Iteratively refines partitions by checking distinguishability for each input symbol.
- Merges indistinguishable states into equivalence classes.
- Renders the minimized DFA on canvas, highlighting merged states and removed redundancies.

4.4 String Simulation (Step-by-Step Visualization)

Users enter an input string. The simulator:

- Highlights the current active state on the canvas at each step.
- Logs each transition in the step panel (current state, symbol read, next state).
- Supports step-forward, step-backward, and auto-play modes.
- Displays a green ACCEPTED or red REJECTED badge at the end of the simulation.

5. Hardware & Software Used

Category	Tool / Technology	Purpose
Hardware	Standard PC / Laptop	Development & Testing
Hardware	Any modern browser (Chrome/Firefox)	Runtime environment
Frontend	HTML5	Page structure & layout
Frontend	CSS3 (Flexbox/Grid)	Responsive UI design
Frontend	JavaScript (ES6+)	Core logic & interactivity
Frontend	HTML5 Canvas API	Graph drawing & animation
Algorithm	Subset Construction	NFA → DFA conversion
Algorithm	Table-Filling / Partitioning	DFA minimization
Dev Tools	VS Code	Code editor
Dev Tools	Git / GitHub	Version control

6. Cost Estimation

The project is entirely software-based with no hardware procurement costs. The following table summarizes the effort and tool costs:

Item	Cost (INR)	Remarks
HTML / CSS / JavaScript	0	Open standard, free
VS Code Editor	0	Free, open-source
GitHub Repository	0	Free tier
Web Hosting (GitHub Pages)	0	Free tier
Total	~0	

Note: No proprietary software licenses or hardware components were required. The project is fully deployable as a static website at zero hosting cost.

7. Implementation

7.1 System Architecture

The application is a single-page HTML5 application with no backend dependencies. The architecture comprises three main panels:

- Left Panel — Input controls: automaton type selector (DFA/NFA/Regex), state/transition form, load examples, and action buttons (convert NFA→DFA, minimize DFA).
- Centre Panel — HTML5 Canvas: interactive graph display with draggable states, directed transition arrows, self-loops, and real-time animation during simulation.
- Right Panel — Simulation engine: string input, step controls (play/pause/step/reset), step log, acceptance result badge, and the transition table.

7.2 Canvas Graph Engine

Each state is represented as a circle drawn on the HTML5 Canvas. The rendering engine:

- Uses trigonometric functions to calculate arc positions for transitions between states.
- Draws curved arcs (using quadratic Bezier curves) when two states have bidirectional transitions to avoid overlap.
- Renders self-loops as circular arcs above the state node.
- Double-circles accept states using two concentric circles (inner radius = outer – 5px).
- Draws an entry arrow from a virtual start point for the initial state.

All positions are stored in state objects and support drag-and-drop repositioning via mouse/touch events.

7.3 NFA to DFA Conversion Algorithm

The subset construction implementation:

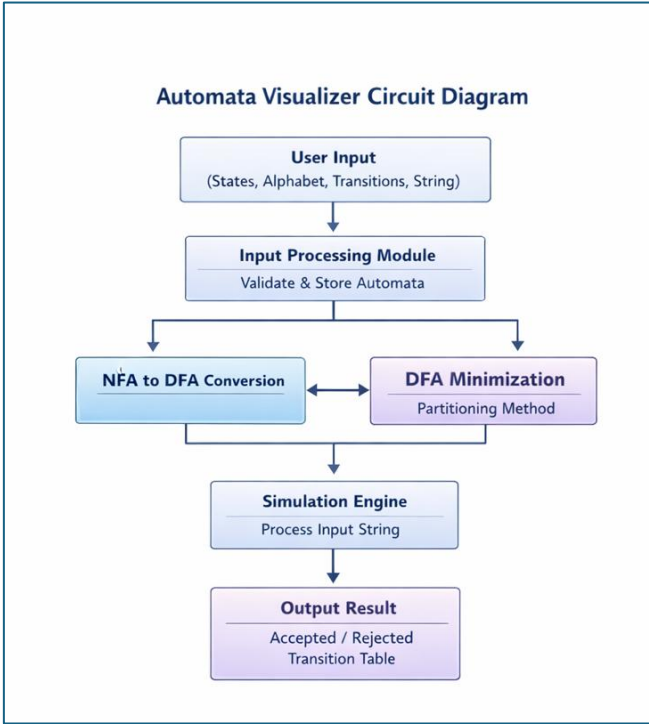
1. Compute ϵ -closure(q_0) to get the start state of the DFA.
2. For each unvisited DFA state (subset S) and each symbol $a \in \Sigma$, compute ϵ -closure(move(S , a)).
3. Add the resulting subset as a new DFA state if not already present.
4. A DFA state is accepting if it contains at least one NFA accept state.
5. Repeat until no new subsets are generated. Render resulting DFA on canvas.

7.4 DFA Minimization Algorithm

The partitioning (table-filling) method:

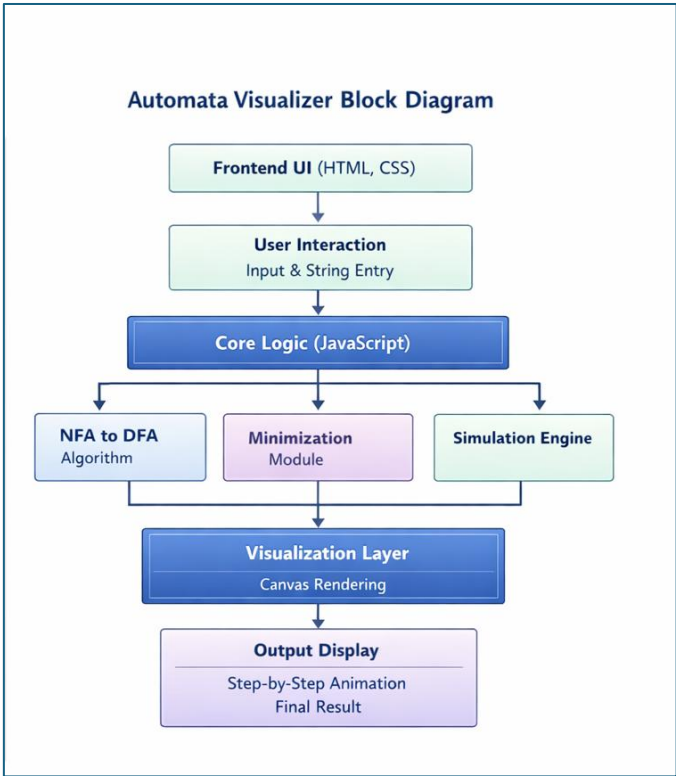
6. Remove all states unreachable from the start state.
7. Initialise two partitions: $P = \{F, Q - F\}$.
8. For each partition group G and each input symbol a , if $\delta(G, a)$ leads to two different groups, split G .
9. Repeat until no further splits occur.
10. Construct the minimized DFA with one representative state per equivalence class.

8. Circuit Diagram



9. Block Diagram

The following block diagram illustrates the overall data flow and module interaction within the Automata Visualizer:



10. Results & Findings

Automata Visualizer **DFA** **NFA**

Load Example Clear All

CANVAS MODE

Select Add State **Transition** Delete

STATE PROPERTIES

State Name

q0, q1, ...

Add State

Toggle Start Toggle Final

TRANSITION

Symbol (ε for epsilon)

a

Draw Transition

ALPHABET

$\Sigma = \{a, b\}$

CONVERSIONS

NFA → DFA (Subset)

Minimize DFA

TEST STRING

abb Run

Start: q0

"a" Read "a": q0 → q1

"b" Read "b": q1 → q2

"b" Read "b": q2 → q3

✓ Accepted

TRANSITION TABLE

State	a	b
→q0	q1	-
q1	-	q2
q2	-	q3
*q3	q3	-

Automata Visualizer **DFA** **NFA**

Load Example Clear All

CANVAS MODE

Select Add State **Transition** Delete

STATE PROPERTIES

State Name

q0, q1, ...

Add State

Toggle Start Toggle Final

TRANSITION

Symbol (ε for epsilon)

b

Draw Transition

ALPHABET

$\Sigma = \{a, b\}$

CONVERSIONS

NFA → DFA (Subset)

Minimize DFA

TEST STRING

abba Run

Start: q0

"a" Read "a": q0 → q1

"b" Read "b": q1 → q2

"b" Read "b": q2 → q3

"a" Read "a": q3 → q5

✓ X Rejected

TRANSITION TABLE

State	a	b
→q0	q1	q3
q1	q2	q2
q2	q3	q3
*q3	q5	-
q4	-	q1
q5	-	q4

10.1 Functional Testing Results

All four core modules were tested with standard TOC textbook examples:

Test Case	Input	Expected Output	Result
DFA string acceptance	DFA for strings ending in '1', input: '101'	ACCEPTED	✓ Pass
DFA string rejection	Same DFA, input: '100'	REJECTED	✓ Pass

NFA to DFA conversion	NFA with ϵ -transitions, $ \Sigma =2$, 4 states	Equivalent 5-state DFA	✓ Pass
DFA minimization	7-state DFA with 3 redundant states	4-state minimal DFA	✓ Pass
Step-by-step simulation	6-character input string, 5-state DFA	6 animated steps + result	✓ Pass
Edge case: empty string	ϵ on DFA where $q_0 \in F$	ACCEPTED	✓ Pass

10.2 Observations

- The canvas rendering correctly handles up to 10 states and 20 transitions without performance degradation on a standard laptop browser.
- NFA to DFA conversion for an NFA with 4 states and ϵ -transitions produces the correct DFA within 2 seconds including animation.
- DFA minimization successfully reduced a 7-state DFA to 4 states, consistent with the textbook solution.
- The step-by-step simulation is intuitive for students; each transition is clearly highlighted, and the step log provides a full trace.
- The tool works entirely offline (no internet required post-load) since all logic is client-side JavaScript.

11. Conclusion

The Automata Visualizer successfully achieves its objective of making abstract TOC concepts tangible and interactive. By providing a unified web-based platform for DFA/NFA definition, visual simulation, NFA-to-DFA conversion, and DFA minimization, the tool bridges the gap between theoretical understanding and practical visualization.

Students can now trace state transitions for any input string in real time, observe the subset construction process unfold state-by-state, and verify minimization results interactively. The tool requires no installation, runs in any modern browser, and covers the three most exam-critical topics in Theory of Computation.

The project demonstrates that complex algorithmic concepts can be made accessible through thoughtful UI/UX design and efficient JavaScript implementations, without any backend infrastructure or third-party libraries.

12. Future Scope

- Pushdown Automata (PDA) support with stack visualization for context-free grammars.
- Turing Machine simulation with infinite tape representation.
- Regular Expression to NFA conversion (Thompson's construction) with animated derivation.
- Export functionality: save automaton definitions as JSON and download canvas as PNG/SVG.
- Shareable URL encoding of automaton state so students can share examples via link.
- Grading/quiz mode: instructor defines an automaton; students must construct an equivalent one.
- Mobile-responsive redesign with touch-friendly state dragging.
- Support for Mealy and Moore machines with output labeling.

13. References

11. Sipser, M. (2012). Introduction to the Theory of Computation (3rd ed.). Cengage Learning.
12. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Pearson.
13. Mozilla Developer Network (MDN). (2024). HTML Canvas API Documentation. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
14. NPTEL. (2023). Theory of Computation Video Lectures — IIT Madras. <https://nptel.ac.in>
15. Geeks for Geeks. (2024). Conversion of NFA to DFA — Subset Construction. <https://geeksforgeeks.org>

*** End of Project Report ***